

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)

Кафедра инструментального и прикладного программного обеспечения (ИиППО)

ОТЧЁТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №3

по дисциплине «Программирование на языке Джава»

Выполнил студент группы ИКБО-70-25 **Мкртчян Г.Г.**

Принял Ермаков С.Р.
Старший преподаватель

Москва 2025

Задача 3.1.1

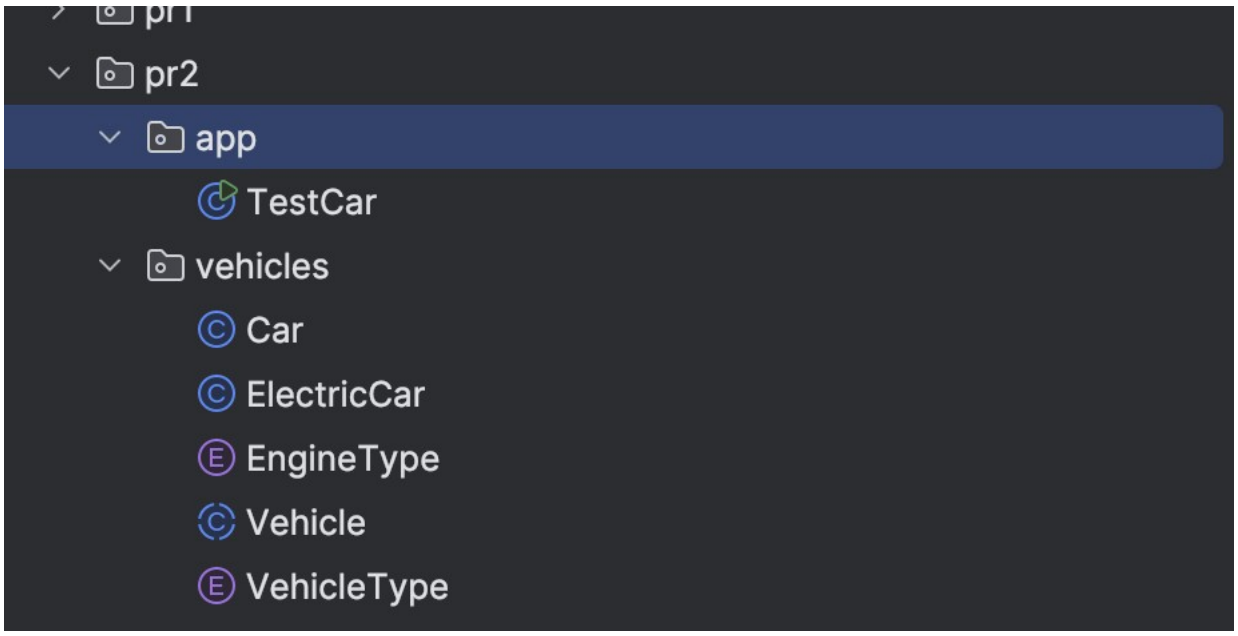
Описание задачи, цель

Определите интерфейс ElectricVehicle, который будет описывать специфические методы для электромобилей. В интерфейсе должны быть следующие методы: getBatteryCapacity() и setBatteryCapacity().
Класс ElectricCar должен наследовать класс Car и реализовывать интерфейс ElectricVehicle. Реализуйте методы интерфейса и добавьте поле batteryCapacity для хранения информации о емкости батареи. В конструкторе задайте тип двигателя как "Electric".

Требования к тестированию программы:
• Созданы экземпляры классов Car и ElectricCar.
• Изменение года выпуска и имени владельца.
• Изменение страхового номера.
• Получение информации о емкости батареи у электромобиля.
• Вывод информации о транспортных средствах в консоль с помощью метода toString().

Ход работы

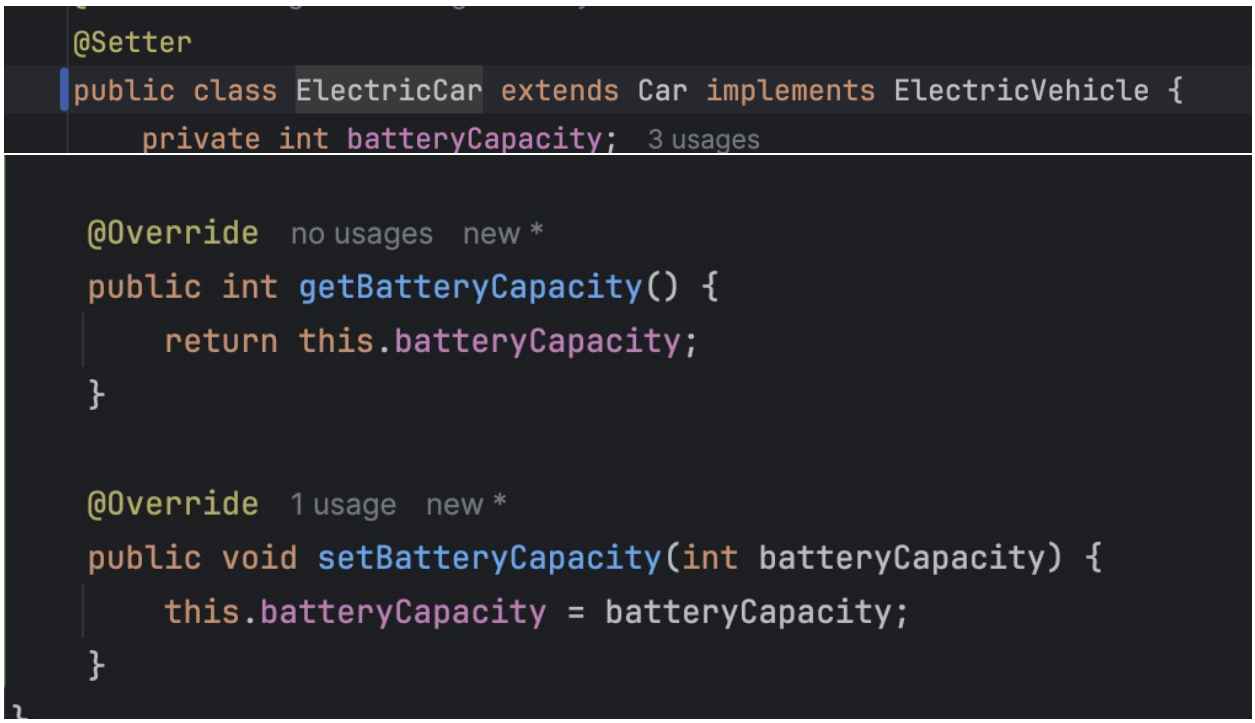
Берем классы из прошлой работы



Создаем интерфейс ElectricVehicle

```
public interface ElectricVehicle {  1 usage  1 implementation
    int getBatteryCapacity();  no usages  1 implementation
    void setBatteryCapacity(int batteryCapacity);  1 usage  1 implementation
}
```

Добавляем реализацию интерфейса ElectricVehicle



Результаты и выводы
Научились создавать интерфейсы

Задача 3.2.1

Описание задачи, цель

Ход работы

Ответы на вопросы по проекту bikeproject

1. Примитивный тип данных для хранения поля класса

int. Примеры: private int NumGears; (Bike), private int frameSize; (MountainBike), private int tyreWidth, postHeight; (RoadBike). String — не примитивный, а ссылочный тип.

2. Конкатенация строк

Bike.printDescription(): склеиваются make, handleBars, frame, NumGears, seatType, tyres и символы "\n" оператором +.

MountainBike.printDescription(): склеиваются type, suspension и frameSize.

RoadBike.printDescription(): склеиваются tyreWidth и postHeight.

3. Имена созданных объектов (BikeDriver)

Объект	Класс
bike1	RoadBike
bike2	RoadBike
bike3	MountainBike
bike4	Bike

4. Количество конструкторов

Класс	Кол-во	Конструкторы
Bike	2	без параметров; с 5 параметрами
RoadBike	3	без параметров; (int postHeight); с 7 параметрами
MountainBike	2	без параметров; с 8 параметрами
BikeDriver	0	явных нет (только main), компилятор добавляет конструктор по умолчанию

5. Наследование

Суперкласс: Bike.

Подклассы: RoadBike extends Bike и MountainBike extends Bike.

Подклассы вызывают super(...) и переопределяют printDescription(), вызывая super.printDescription() и добавляя свою строку.

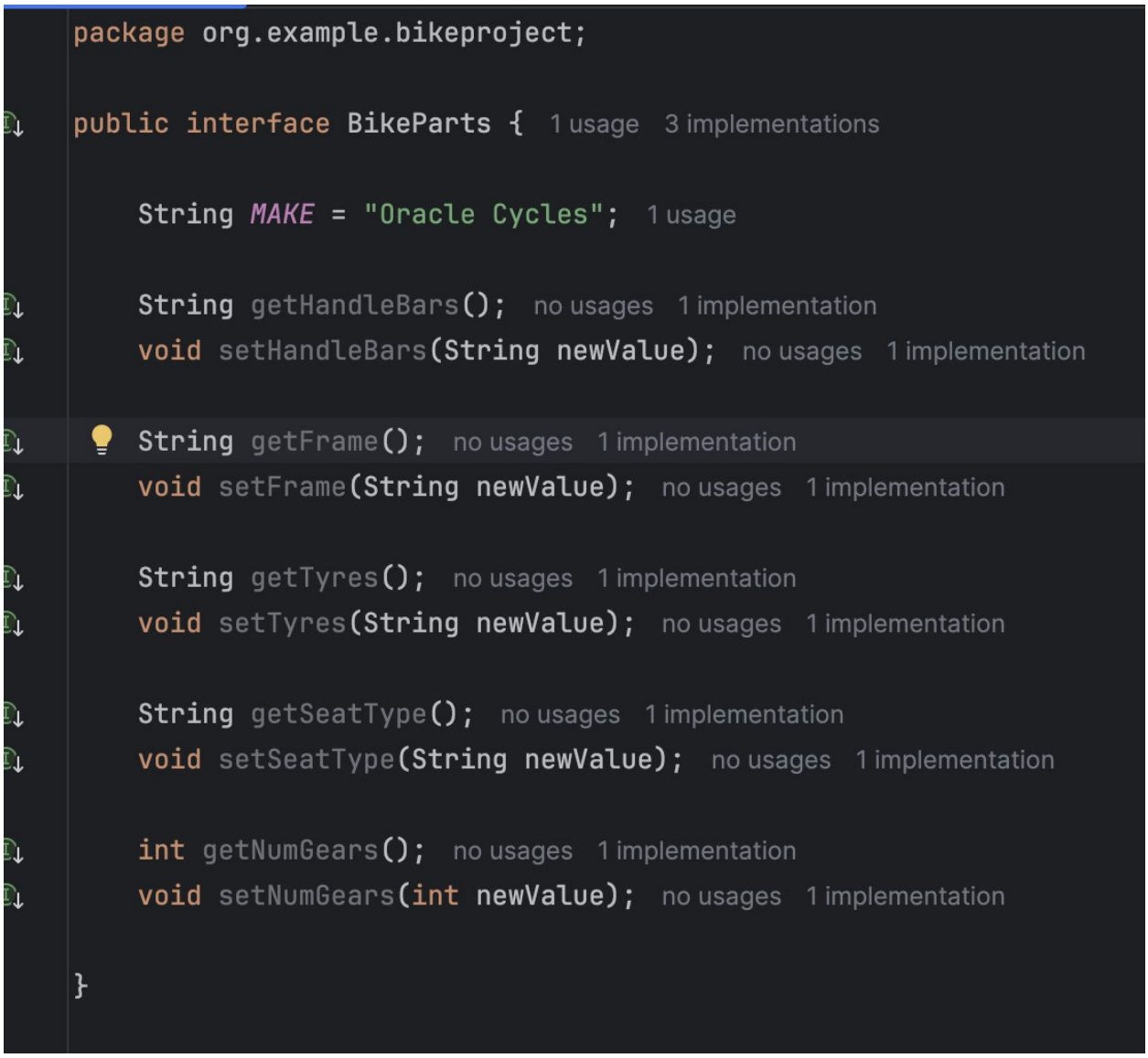
6. Стандартные велосипеды (значения по умолчанию)

Значения взяты из конструкторов без параметров.

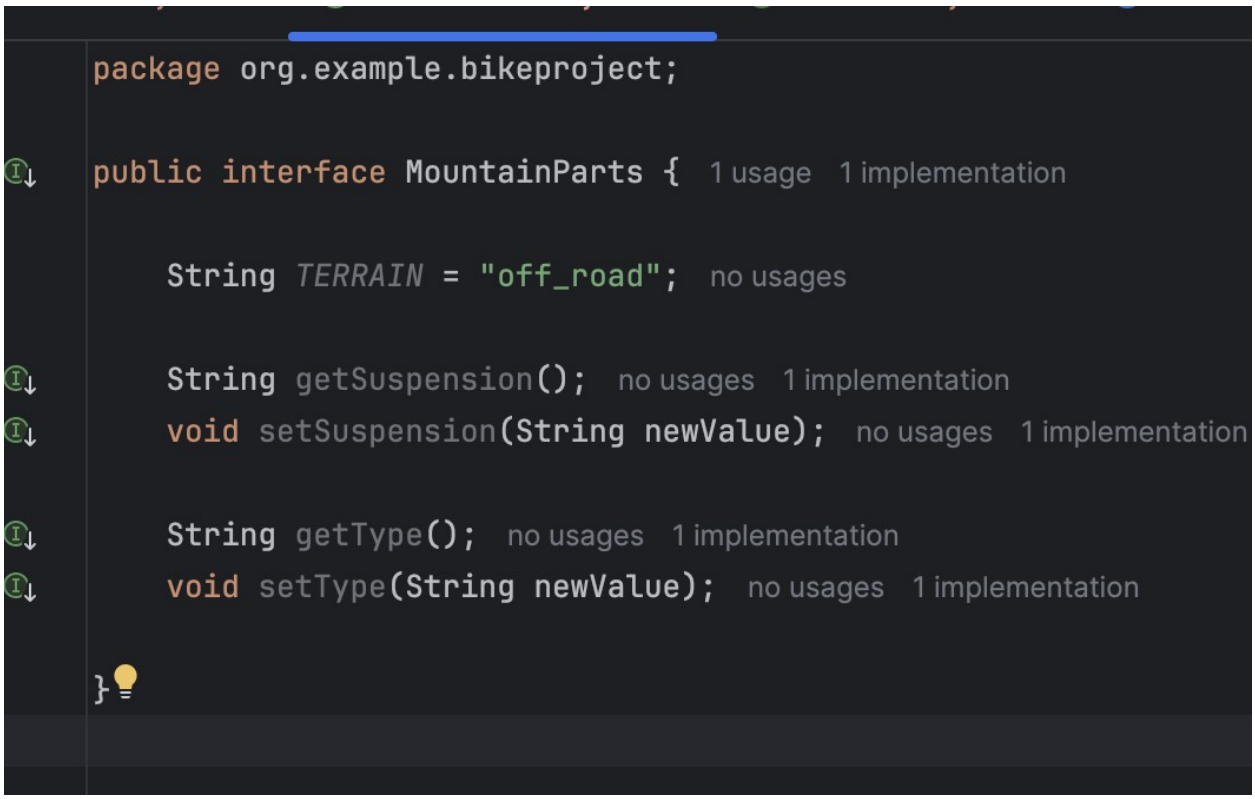
Поле	Стандартный RoadBike (bike1)	Стандартный MountainBike (bike3)
handleBars	"drop"	"Bull Horn"
frame	"racing"	"Hardtail"
tyres	"tread less"	"Maxxis"
seatType	"razor"	"dropper"
numGears	19	27
tyreWidth	20	—
postHeight	22	—
suspension	—	"RockShox XC32"
type	—	"Pro"

frameSize	—	19

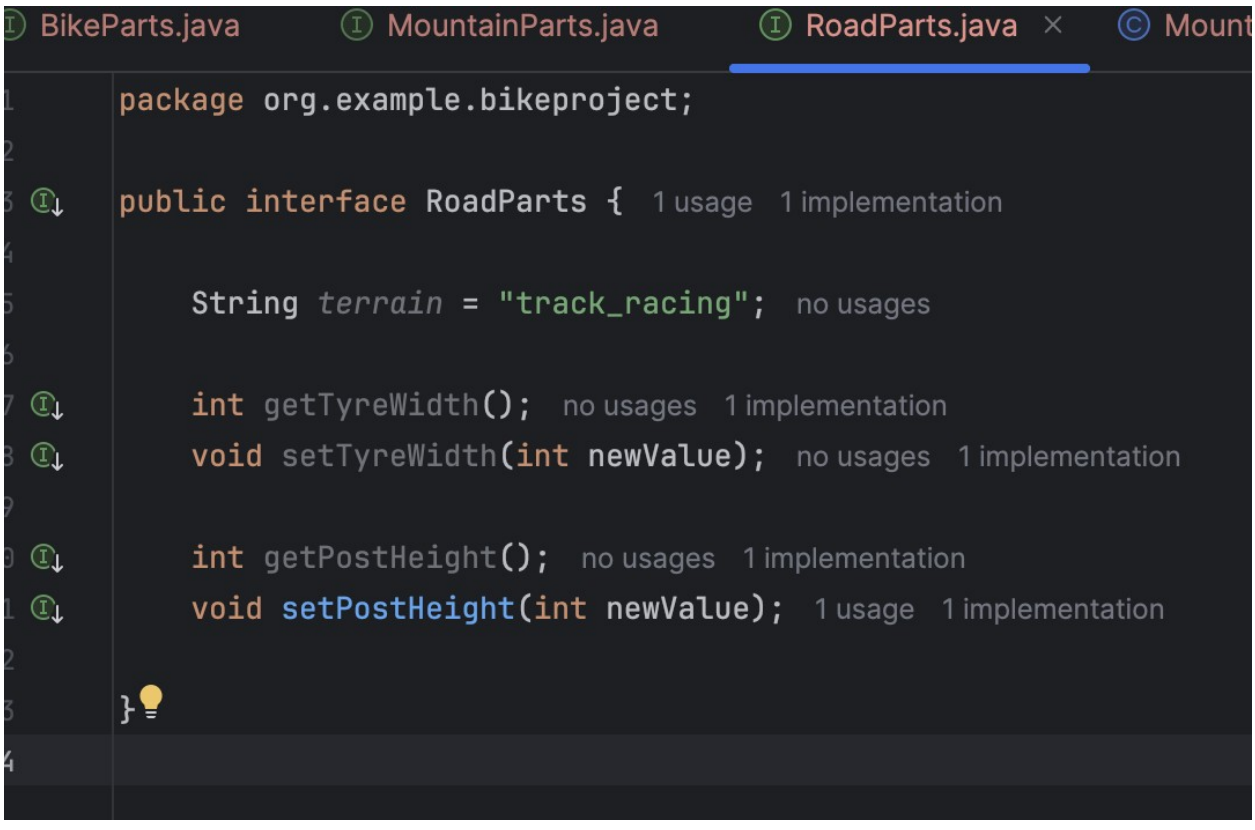
Создаем интерфейс с константным производителем велосипедов и методами



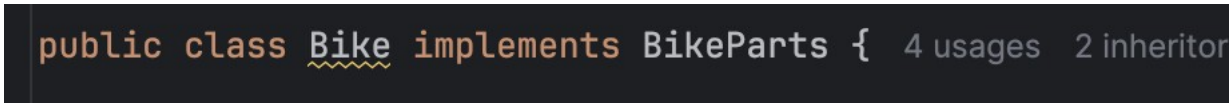
Создаем интерфейс MountainParts:



Создаем интерфейс RoadParts



Доюавляем реализацию интерфейса BikeParts классу Bike



```
public String getHandleBars() { return handleBars; }  no usages
public void setHandleBars(String newValue) { handleBars = newValue; }  no usages

public String getFrame() { return frame; }  no usages
public void setFrame(String newValue) { frame = newValue; }  no usages

public String getTyres() { return tyres; }  no usages
public void setTyres(String newValue) { tyres = newValue; }  no usages

public String getSeatType() { return seatType; }  no usages
public void setSeatType(String newValue) { seatType = newValue; }  no usages

public int getNumGears() { return NumGears; }  no usages
public void setNumGears(int newValue) { NumGears = newValue; }  no usages
```

Добавляем реализацию MountainParts классу MountainBike:

```
public class MountainBike extends Bike implements MountainParts {  2 usages
```

```
}//end method printDescription
public String getSuspension() { return suspension; }  no usages
public void setSuspension(String newValue) { suspension = newValue; }  no usages

public String getType() { return type; }  no usages
public void setType(String newValue) { type = newValue; }  no usages
```

Добавялем реализацию RoadParts классу RoadBike:

```
public class RoadBike extends Bike implements RoadParts {  4 usages
```

```
public int getTyreWidth() { return tyreWidth; }  no usages
public void setTyreWidth(int newValue) { tyreWidth = newValue; }  no usages

public int getPostHeight() { return postHeight; }  no usages
public void setPostHeight(int newValue) { postHeight = newValue; }  1 usage
```

Обновляем высоту столба для bike1 до 20 и запускаем

```
bike1.setPostHeight(20);
System.out.println("\nUpdated bike1:");
bike1.printDescription();
```

```
Oracle Cycles
This bike has drop handlebars on a racing frame with 19 gears.
It has a razor seat with tread less tyres.
This Roadbike bike has 20mm tyres and a post height of 22.

Oracle Cycles
This bike has drop handlebars on a tourer frame with 14 gears.
It has a comfort seat with semi-grip tyres.
This Roadbike bike has 25mm tyres and a post height of 18.

Oracle Cycles
This bike has Bull Horn handlebars on a Hardtail frame with 27 gears.
It has a dropper seat with Maxxis tyres.
This mountain bike is a Pro bike and has a RockShox XC32 suspension and a frame size of 19inches.

Oracle Cycles
This bike has null handlebars on a null frame with 0 gears.
It has a null seat with null tyres.

Updated bike1:

Oracle Cycles
This bike has drop handlebars on a racing frame with 19 gears.
It has a razor seat with tread less tyres.
This Roadbike bike has 20mm tyres and a post height of 20.
```

Результаты и выводы

Создавали интерфейсы для всех классов и определяли необходимые методы